VERIQO Backup and Restore Procedure

The durable Commercial Store keeps its state in a write-ahead log directory, selected at startup with `--commercial-wal-dir=/path/to/wal`. Backup and restore both operate on that directory.

Status: the mechanism is implemented and its round trip is tested (`TestStoreBackupAndRestoreRoundTrip`). **No drill against a live deployment has been performed**, and there is no built-in scheduler. Both are deployment work, and §7 says how to close them.

---

1. The design, and why it matters operationally

Every mutating call is appended to the WAL **before the caller is told it succeeded**. A caller who received a success response has a durably logged operation, not merely an in-memory one.

On startup, the store replays those recorded inputs through its own real logic and reconstructs byte-identical state — the same hashes, not merely equivalent-looking data.

Restore uses the identical code path as ordinary crash recovery. A backup is not a separate, less-tested format; it is a copy of the real thing, replayed by the real recovery routine. This is the single most important property of this procedure: the restore path is exercised every time any deployment restarts.

---

2. Taking a backup

`Store.Backup(destDir)`:

1. Flushes the WAL (fsyncing the open segment), so the copy includes every record any caller was ever told was durable.
2. Copies every segment file into `destDir`, which must not already exist.

Returns `ErrNotDurable` for an in-memory store — it will not pretend to back up something that has nothing on disk.

Filesystem-level alternative. Copying the WAL directory with ordinary filesystem tools also works and is what a snapshot-based backup system will do. Two cautions:

- A copy taken while writes are in flight may capture a partially written tail record. This is *safe* — recovery detects and truncates it, reporting the defect — but that tail operation will be absent from the restored store.
- Copy the whole directory. A partial set of segments is a truncated history, and will restore as one.

---

3. Restoring

`RestoreStoreFromBackup(backupDir, walDir)` copies the backup into a fresh WAL directory and opens it exactly as any other WAL directory, returning the reconstructed store and a recovery report.

Operationally:

```
# 1. Stop the gateway. Do not restore over a running deployment.
# 2. Move the existing WAL directory aside -- never delete it.
mv /var/lib/veriqo/wal /var/lib/veriqo/wal.pre-restore.$(date +%s)
```

```
# 3. Restore into a fresh directory, then start against it.
veriqo-gateway --commercial-wal-dir=/var/lib/veriqo/wal   # after copying backup in
```

Never restore into an existing WAL directory, and never delete the directory you are replacing. If the restore turns out to be from the wrong point in time, the directory you moved aside is the only way back.

---

4. Reading the recovery report

Startup produces a recovery report. Check it every time — it is the system telling you what it found on disk.

Field	Meaning
`RecoveredRecords`	Records read from disk.
`ReplayedRecords`	Records successfully replayed into state.
`LostRecords`	Records that could not be recovered. **Any non-zero value needs investigation.**
`FailedClosed`	The log refused to open rather than proceed with a defect it would not silently accept.
`Findings`	Classified defects: `PartialRecord`, `CRCFailure`, `TornWrite`, `CorruptTail`, `CorruptMiddle`, `ChainBroken`, `BadMagic`, `ImplausibleSize`.

A corrupt *tail* is normal after a crash — the last write was in flight. It is truncated and reported, and the operation it represented did not complete.

A corrupt *middle* or a broken chain is not normal. It indicates damage to history rather than to the in-flight write. Treat as SEV-1 and follow the Incident Response Procedure — preserve before remediating.

A foreign or unrecognizable record fails the restore outright rather than being skipped: a corrupted log must never be mistaken for a smaller, valid one.

---

5. Verify the restore before returning to service

A restore that starts is not a restore that worked. Before putting the deployment back in rotation:

1. Recovery report shows `LostRecords: 0` and no middle/chain findings.
2. `GET /readyz` returns 200.
3. Fetch several known cases and confirm they are present and decided.
4. **Run replay on several cases and confirm `converged: true`.** This is the substantive check — it proves the reconstructed state produces byte-identical hashes, not merely that records exist.
5. Verify an evidence item (`POST /v1/evidence/{id}/verify`).
6. If you hold a dossier package exported before the incident, verify it with the standalone verifier and confirm the restored store's hashes for that case still match it. This is the strongest available end-to-end confirmation.

---

6. RPO and RTO

RPO (data loss window) is determined by backup frequency, not by the software. Between backups, the WAL on the live host is the only copy — so a host-loss event loses everything since the last backup. If your RPO is one hour, back up hourly; nothing in VERIQO makes that unnecessary.

Within a single host, RPO for a *process* crash is effectively one in-flight operation: everything acknowledged is durable.

RTO is dominated by replay time, which scales with the number of recorded operations. Measure it on your own data volume during the pilot rather than assuming — this is exactly what the drill in §7 is for.

---

7. What to establish during a pilot

These are the named open items; closing them is deployment work, not development:

- ☐ **Automate backups.** No scheduler is built in. Use `Backup` on a timer, or a filesystem/volume snapshot regime.
- ☐ **Store backups off-host.** A backup on the same disk as the WAL protects against corruption, not against losing the host.
- ☐ **Run a real restore drill.** Restore into a separate environment and complete §5 in full. Record the elapsed time — that is your measured RTO.
- ☐ **Write your own runbook** with your real paths, hostnames, and escalation contacts. This document is the mechanism; a runbook is the deployment-specific procedure.
- ☐ **Set retention for the backups themselves**, and confirm it does not conflict with any legal hold in force on the evidence they contain.
- ☐ **Encrypt backups at rest.** The application does not encrypt WAL segments, so neither will a copy of them.